Orange Visual Condition Classification

Data Science assignment adaptation | Two-model computer vision comparison

Scope clarification: The original Greenpod Labs assignment requested **avocado ripening stages**. The supplied dataset contains **orange appearance/condition labels**. At the candidate's request, this is an **orange-only adaptation**, and it must not be represented as completed avocado-ripeness work.

1. Problem definition and actual class labels

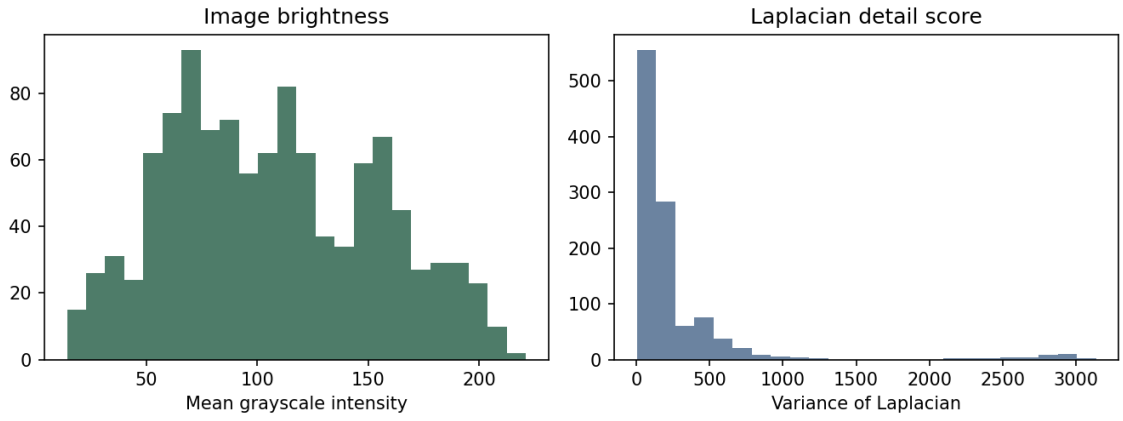
Input: a photograph labeled as an orange. Output: one of four provided folder classes: **blackspot**, **canker**, **fresh** or **grenning**. The last name is preserved exactly as spelled in the source. This project classifies a single image; the optional green candidate bounding rectangle in the demo is **heuristic localization**, not a trained object detector.

2. Dataset inspection, outliers and biases

The uploaded dataset contains **1,090** readable images before exact-duplicate removal. Per-class totals: blackspot (206), canker (201), fresh (314), grenning (369).

Largest/smallest class ratio: **1.84:1**. Automated quality screens flagged **72** dark, **2** bright, **354** low-detail, **0** small, **0** byte-duplicate instances and **6** possible perceptual-hash collisions. These are **review flags, not confirmed bad images**. Unreadable images: **0**.





Observed background bias: a manual contact-sheet check showed several **fresh** samples with white graphic-like backdrops and attached leaves, while sampled **grenning** images have dark backgrounds. Source composition, lighting, framing, and lesion severity may therefore correlate with labels. Apparent high accuracy could partly reflect

these artifacts rather than biological symptoms. The folder label **grenning** may be a spelling error; do not silently relabel it. Inspect label semantics and class boundaries with the dataset owner. Multiple photos of the same fruit may produce overoptimistic estimates if a fruit identifier is unavailable.

3. Data splitting, preprocessing and reproducibility

The original training and test folders are respected. A stratified validation subset is created from the training folder; the provided test folder remains a final untouched evaluation holdout. Exact SHA-256 duplicate copies are removed before model training and split assignment. Images used by the CNN are RGB, uniformly resized to **96 x 96**, normalized, and augmented **only during training** using mild flips, rotations and color jitter.

Class	Training	Validation	Test
blackspot	152	32	22
canker	147	32	22
fresh	231	50	33
grenning	286	61	22

All splits and image hashes are recorded in results/split_manifest.csv. The original test set is not used for hyperparameter tuning; near-duplicate photographic views may still exist.

4. Approach A - handcrafted color/texture features + SVM

Color histograms are extracted from HSV and CIELAB channels (24 bins per channel), together with a local binary pattern (LBP) histogram and grayscale texture/contrast statistics. Features are calculated primarily inside a central oval region to reduce - but not eliminate - background effects. A StandardScaler and class-weighted radial-basis-function Support Vector Machine form the classifier.

Why this approach? It is an interpretable, low-data baseline: color and textured surface patterns may help distinguish visible lesion/blemish classes. Weaknesses include fixed hand-crafted features, viewpoint sensitivity, and limited understanding of localized lesions.

5. Approach B - compact convolutional neural network

A small four-block CNN is trained on RGB images using AdamW and class-weighted cross-entropy. Only training images receive augmentation. The checkpoint with best validation loss is retained, with early stopping after repeated lack of improvement. **It was trained from scratch; pretrained weights are not claimed.**

Why this approach? A CNN can learn spatial combinations of color, local lesion shape, pattern and texture without explicitly coding each feature. A compact architecture is practical for this dataset and a CPU-only environment, though it may underperform pretrained transfer learning and can overfit on small datasets.

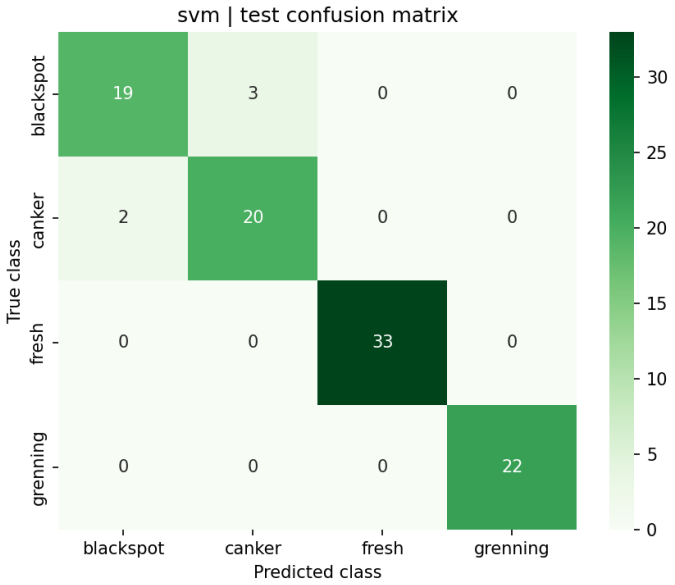
6. Metrics, model comparison and final selection

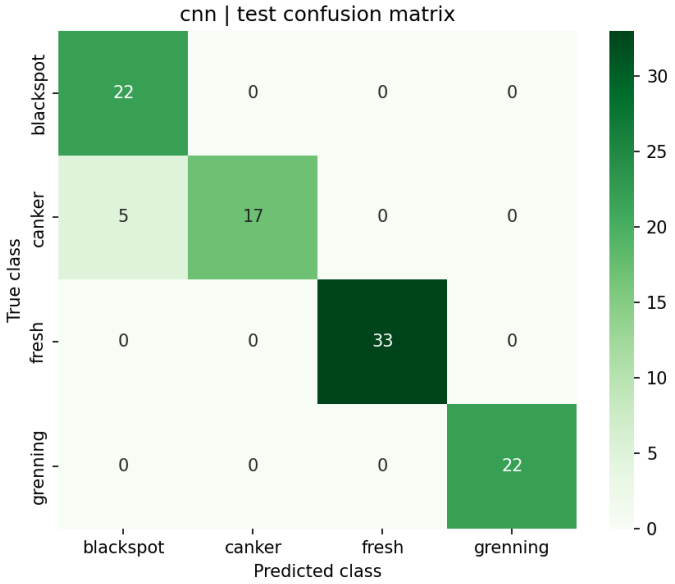
Accuracy summarizes correct predictions but can be dominated by frequent classes. **Macro precision** equally weighs false-positive patterns across four classes, **macro recall** equally weighs missed classes, and **macro F1** balances precision and recall across classes. **Balanced accuracy** averages class recall. Macro F1 is the primary validation selection metric because the orange classes are not perfectly balanced.

Model / subset	Accuracy	Macro Precision	Macro Recall	Macro F1	Balanced Acc.
svm / val	0.960	0.950	0.945	0.945	0.945
cnn / val	0.931	0.923	0.906	0.904	0.906

Model / subset	Accuracy	Macro Precision	Macro Recall	Macro F1	Balanced Acc.
svm / test	0.949	0.944	0.943	0.943	0.943
cnn / test	0.949	0.954	0.943	0.942	0.943

Model chosen by validation macro F1: svm. Its final held-out test accuracy is **94.9%**, and test macro F1 is **0.943**. This is an **offline within-dataset evaluation**, not production accuracy.





svm class-wise test recall: blackspot: 86.4%, canker: 90.9%, fresh: 100.0%, grenning: 100.0%.

cnn class-wise test recall: blackspot: 100.0%, canker: 77.3%, fresh: 100.0%, grenning: 100.0%.

7. One-image inference demonstration

Held-out example: orange_blackspot_example.jpg; actual folder label **blackspot**. SVM predicted **blackspot** (score 0.899); cnn predicted **blackspot** (softmax score 0.830). These scores are **uncalibrated** and should not be treated as validated probabilities.



8. Mandatory questions and answers

Q1. In production how does your model perform?
This system has **not been deployed in production**. Its measured results are from the held-out orange dataset shown above. In a controlled setting with similar photography, the measured test performance gives an initial estimate, but different lighting, varieties, mobile cameras or previously unseen disease appearances can reduce reliability. Before deployment, benchmark end-to-end CPU/GPU latency on target hardware, evaluate samples from new farms and seasons, monitor per-class recall, and introduce manual review for uncertain cases.

Q2. What is the drawback of your model?
The baseline depends on fixed color/texture features; the CNN has limited data and is trained from scratch. Background cues, class ambiguity and lighting may cause false predictions. The system assigns one label per image, so mixed infections or multiple fruit objects cannot be described reliably. The demo rectangle is heuristic, not a trained orange detector. Since fruit/plant IDs are unavailable, multiple photos of a single source may leak similarity across splits.

Q3. If given more time, what would you improve?
Audit suspicious samples and source labels with a domain expert; obtain more balanced images from new orchards, lighting conditions and devices; group splits by fruit or acquisition batch; use additional class-sensitive metrics and confidence calibration; compare a pretrained MobileNetV2, experiment with stronger segmentation or object detection for cluttered scenes, and benchmark speed and robustness on representative production hardware.

Q4. Where did you spend more time solving the assignment?
The main technical focus was inspecting the supplied dataset, retaining its original holdout split, assessing quality and duplicate risks, establishing a fair comparison between feature-based SVM and CNN, and interpreting per-class errors. The CNN training and evaluation also required repeatable preprocessing and checkpointing. This describes the implemented workflow, not a fabricated personal time log.

9. Flask demonstration interface

A small **Flask** web application connects the two saved models to an image-upload page. The user selects a photo, clicks Predict, and sees the SVM and CNN outputs side by side. The page also displays measured test results and confusion matrices from the saved files. This interface demonstrates inference; it does not provide evidence of field-ready performance. Run it locally with **python app.py** after installing requirements-web.txt.